

How a Proof Becomes a Program

An unhurried tour of a small formal system in which theorems, once proved, hand you a working program for free — and of what that turns out to cost.

This companion is written to be read start to finish by someone who has never seen realizability, type theory, ordinals past ω , or a proof assistant. It assumes only that you know what a theorem and a proof are, what the natural numbers $0, 1, 2, \dots$ are, and that you have met mathematical induction at least once. Everything else is built up on the page, and nothing is used before it has been explained. Each new idea here appears at the exact moment an earlier paragraph has forced you to ask for it.

Every factual claim about the formalization traces to the machine-checked development it describes; the fragment, the theorems, and the numbers below are all real and all checked by the Lean proof assistant.

Act I — The hook

Pick a natural number — say 3. Write it down. Now play the following game with it.

First, rewrite the number using only powers of 2, and rewrite the exponents that way too, and their exponents, all the way down, until nothing but 2s and small constants remain. This is called *hereditary base 2*. Now change every 2 you wrote into a 3 — bump the base — and subtract 1 from the result. Then rewrite *that* number in hereditary base 3, change every 3 into a 4, subtract 1 again. Then base 4 into base 5, subtract 1. And so on, bumping the base by one each step and subtracting a single unit each time.

The bases are marching upward without bound. Each step replaces a base by a larger one everywhere it appears — and near the start, that base change inflates the number far more violently than the lonely “subtract 1” ever pulls it back down. The sequence you are generating does not drift gently; for many starting values it erupts. Start at 4 and the first few terms are 4, then 26, then 41, then 60, and it keeps climbing from there into numbers with no short decimal name.

Goodstein’s theorem says that this sequence — no matter which number you started from — always, eventually, comes back down and hits exactly 0.

That is already strange. A process whose every early move makes the number enormously bigger, driven downward only by subtracting one at a time, nonetheless terminates at 0 for

every starting value. But the strange part, for our purposes, is not the theorem itself. It is what it takes to prove it, and what you can do with the proof once you have it.

Here is the first half of the surprise. You cannot prove Goodstein’s theorem with ordinary mathematical induction over the natural numbers. The usual “check 0, then check that each number’s truth follows from the previous one” is genuinely not enough; the theorem needs a stronger principle, one that reaches past where step-by-step induction on $\mathbb{N}$ can go. (Exactly what that stronger principle is, and why it is stronger, is Act IV’s business. For now, hold onto the fact that ordinary induction falls short.)

Here is the second half. Despite needing that extra strength, the proof is *constructive* — it does not merely assert that the sequence reaches 0, it contains, buried inside it, a recipe for finding the exact step at which that happens. And in the formal system this tour is about, that recipe can be pulled out of the proof mechanically and run as an actual program. Feed it a starting number; it returns the stopping step.

Most expositions of Goodstein’s theorem stop at the statement, or gesture at the proof and leave the “constructive content” as a phrase. This one opens the box all the way. We are going to watch, in complete detail and on a real example, a proof turn into a program — first on something tiny enough to hold in your hand, then on Goodstein itself, and then on a handful of other theorems that each ask the machine to do one genuinely new thing. By the end you will have seen the whole mechanism, and one question will remain: when a program falls out of a proof this powerful, does some sliver of non-constructive reasoning sneak into the program along the way? The last act is devoted to answering it.

Act II — One tiny, real example, walked through completely

Before any general theory, one concrete thing, done slowly and in full. We will not introduce a single piece of machinery in this act beyond what this one example forces us to name. No hierarchies, no levels, no collapses — those are Act III, and they will make far more sense once you have a real realizer in hand to point at.

Everything in this tour happens inside a fixed, small formal system — we will call it *the fragment*. Think of the fragment as a self-contained little world of mathematics with a rigid rulebook: it has a fixed vocabulary of things it can talk about (zero, successor, addition, multiplication, and a modest list of further operations we will meet as we need them), a fixed grammar of statements it can form, and a fixed list of inference rules for building proofs. Nothing enters the fragment except through that rulebook. This rigidity is deliberate: because the rules are finite and mechanical, a proof in the fragment is itself a finite, mechanical object — a tree of rule-applications — and a machine can take that tree apart. Hold that thought; it is what makes everything later possible.

Our example is the fragment’s very first existential theorem — the first time it proved that *something exists* rather than that two things are equal. It is deliberately the warm-up the development itself used before tackling the general Goodstein theorem, and we use it for exactly that reason: in Act IV, Goodstein will be the grown-up version of this same statement, and

you will already have watched the miniature happen by hand.

The statement is:

There is a step s at which the Goodstein sequence starting from 3 has reached 0.

In the fragment’s grammar this is written

$\exists s. \text{good}(3, s) = 0$

where $\text{good}(3, s)$ is the fragment’s own name for “the value of the Goodstein sequence started at 3, after s steps.” Read the whole formula aloud: *there exists a step s such that $\text{good}(3, s)$ equals 0.*

We happen to know the answer. The Goodstein sequence starting from 3 runs

3, 3, 3, 2, 1, 0

— so it first reaches 0 at step 5. The number 5 is the *witness*: the particular s that makes $\exists s. \text{good}(3, s) = 0$ true. Keep 5 in mind; the entire act is about watching 5 travel from the proof into a running program and back out.

What a realizer for this has to be

Here is the first idea we genuinely need, and it arrives because the statement forces it. The statement claims something *exists*. A proof that merely convinces us the claim is true — without ever saying *which* s works — would be useless for computing anything. So we ask: what would a proof have to carry, concretely, for us to be able to read the answer off it?

At a minimum, it has to carry the witness — the number 5 — because without the witness there is nothing to return. And it has to carry, as well, some evidence that this particular witness really does the job: that $\text{good}(3, 5)$ really is 0. A witness with no evidence is a guess; a proof is not a guess.

This “what a proof carries” is exactly the notion of a **realizer**. A realizer of a statement is the computational content of a proof of it — the data a proof hands you that a machine can actually use. Different shapes of statement carry different shapes of data, and the shape is dictated entirely by the statement’s outermost connective. For an existential statement $\exists y. \varphi$, the realizer is precisely the pair we just argued for:

A realizer of $\exists y. \varphi$ is a pair: a **witness** (a number), and a **certificate** that φ holds when y is that witness.

That is not a metaphor or a convention chosen for this paper; it is the actual clause the formal system uses to define what “realizes” means at an existential statement. Written a little more precisely, a realizer x of $\exists y. \varphi$ is an object from which you can read a witness — call it w — and which then realizes φ with y set to w . In the development the witness is recovered from x by reading its first component at a fixed “canonical point,” and the rest of x realizes the body. We will not need the canonical-point machinery in detail; what matters is the shape: *witness, then certificate*.

Now specialize this to our formula. The body of our existential is $\text{good}(3, s) = 0$ — an **equation**. And here a second small idea appears, again forced by the example rather than

imposed on it. What does it take to realize an equation? An equation is either true or it is not; if it is true, there is nothing further to *compute* about it — no witness to choose, no case to decide. So the realizer of a true equation carries no information at all. Any object whatsoever realizes it, provided the equation genuinely holds. We call such a realizer **contentless**.

Put the two observations together. A realizer of $\exists s. \text{good}(3, s) = 0$ is a pair whose first component is the witness 5, and whose second component — the certificate that $\text{good}(3, 5) = 0$ — is contentless, because $\text{good}(3, 5) = 0$ is an equation. So the realizer is, in effect, *just the number 5*, wearing a trivial second component it never has to justify beyond the equation being true. All of the content is the witness. That is as simple as a realizer ever gets, which is exactly why we started here.

The proof

Now we build an actual proof of $\exists s. \text{good}(3, s) = 0$ in the fragment, and we build it in the shape our realizer analysis predicts.

To prove an existential, the fragment has a rule — existential introduction — that says: to conclude $\exists y. \varphi$, supply a specific term for y and a proof that φ holds for that term. We supply the term 5. That leaves us needing a proof, inside the fragment, that $\text{good}(3, 5) = 0$.

And this the fragment can simply *compute*. Among its operations is **good** itself, and the fragment’s rules include the defining equations that let it evaluate **good** on concrete numbers, step by mechanical step, until it reaches a numeral. Running those rules on $\text{good}(3, 5)$ grinds out 0, giving a fragment-internal proof that $\text{good}(3, 5) = 0$ — not an appeal to an outside calculation, but a derivation built from the fragment’s own equational rules. (The development calls this computed proof **goodComputeDeriv**; you feed it the start value and the step and it produces the equation.)

So the whole proof is: *existential-introduction, at 5, over the computed proof that $\text{good}(3, 5) = 0$* . It is a small tree with the witness 5 sitting at the branch point. In the development this proof object has a name — **goodThreeExDeriv** — and it is a genuine, closed, machine-checked derivation, not a sketch.

Notice that the proof’s *structure* mirrors the realizer’s structure exactly: the place where the rule supplied the term 5 is the place where the realizer’s witness comes from; the computed sub-proof of the equation is the place where the contentless certificate comes from. This is not a coincidence. It is the mechanism we are here to watch.

What **extract** does to that proof

The fragment comes with a function — call it **extract** — that takes a finished proof and returns its realizer. It is a completely mechanical translation: it walks the proof tree and, at each rule, does the small operation that rule’s realizer calls for. We will see it in generality in Act III; here we only need what it does to *our* proof.

Our proof was existential-introduction at 5 over a computed equation. **extract** handles existential-introduction by building the pair we described: it puts the witness — the number the rule supplied, 5 — in the first component, and it puts the realizer of the sub-proof in the second. The sub-proof was of an equation, whose realizer is contentless, so the second

component is trivial. The result is the pair

$\langle 5, (\text{contentless}) \rangle$

— exactly the realizer our up-front analysis said it had to be. `extract` did not need to be clever. The proof already had the witness in it; all `extract` did was package it in the standard shape for existentials.

Running it

The realizer is not an abstraction; it is an object a machine can evaluate. To get the answer back out, we read its first component at the canonical point — the mechanical “read the witness” operation the existential clause specified. Doing that to $\langle 5, \text{contentless} \rangle$ returns

5

And that is the full round trip, on the smallest possible real example. A statement was formed. We reasoned about what data a proof of it would have to carry, and found: a witness, plus a contentless certificate. We built such a proof by hand, supplying 5. We ran `extract`, which repackaged the proof’s witness into a realizer. We evaluated the realizer, and 5 came back.

Nothing here required knowing what a “level” is, where realizers “live,” or how any of this scales to real theorems. Those questions are about to push themselves on us — the moment we ask “does this work for *every* proof, not just this one?” — and Act III introduces exactly the machinery those questions demand, in the order they demand it.

Act III — Opening the box

We have watched one proof become one program. Two questions are now unavoidable, and the whole theory in this act exists to answer them.

First: our example’s body was an equation, whose realizer was contentless — so *this* realizer was essentially just a number. Real theorems have bodies with `and`, `or`, implications, `for all`. What are realizers of *those*? Until we know, we cannot handle anything but the simplest statements.

Second: we ran `extract` on one proof and got the right answer. Is that luck, or does `extract` produce a correct realizer for *every* proof the fragment can build? If it is only sometimes correct, the whole idea is worthless. We need this to be a theorem.

We take the questions in that order, because the second one needs the first.

Realizers, for every shape of statement

In Act II we met the realizability clause for $\exists$ by reasoning about what a proof of an existential must carry. Every other connective has such a clause, arrived at by the same kind of reasoning, and once you have seen one the rest are recognizable rather than surprising. Here is the full set, each stated as “what data realizes a statement of this shape,” with the same guiding question each time — *what would a machine need to use a proof of this?*

- **An equation** $s = t$ is realized by anything, provided the equation holds. (This is the contentless case from Act II. Equations carry no computational content.)
- **A conjunction** φ and ψ is realized by a **pair** — a realizer of each side, both at once, since using a proof of “both” means getting at each side.
- **A disjunction** φ or ψ is realized by a **tag plus a payload**: a bit that says *which* side holds — 0 for the left, non-zero for the right — and then a realizer of whichever side the tag named. To use a proof of “one or the other,” you first need to know which one; that is the tag. (Look back at $\exists$: its realizer is *witness, then realizer of the body* — structurally the very same “a number, then a payload selected by it” as disjunction’s “tag, then payload.” The existential is disjunction with the two-valued tag replaced by an arbitrary numeral. This is not a slogan; the development literally builds $\exists$ ’s realizer out of disjunction’s pairing machinery. We will return to what that reuse buys us.)
- **An implication** $\varphi \rightarrow \psi$ is realized by a **transformation**: a procedure that, handed any realizer of φ , produces a realizer of ψ . To use a proof of “if φ then ψ ,” you feed it evidence of φ and it gives you evidence of ψ . The realizer *consumes* an input.
- **A universal** $\forall y. \varphi$ is realized by a **procedure indexed by a number**: handed any number k , it produces a realizer of φ with y set to k . To use “for all y ,” you pick a y and it gives you the instance. Like implication, it *consumes* an input — here, the number you instantiate at.

You have now seen the shape of every realizer the fragment will ever produce. Two of them — implication and universal — take something in and give something back; they are the procedures, the parts that *do work* at runtime. The others — pair, tag, witness — just *hold* data. Keep that distinction in view; the next section — levels — is built entirely on it.

Levels: the cost of consuming

Here is the question the previous paragraph forced. Implications and universals are procedures — they take an input and return an output. But that input is *itself* a realizer, which might itself be a procedure taking an input, and so on. A realizer can be a procedure whose inputs are procedures whose inputs are procedures. How deep does this nesting go, and how do we keep track of it?

The fragment keeps track with a single number attached to each formula, called its **level**. The level counts how many layers of “takes an input and returns an output” a realizer of that formula can involve. The counting rule is short, and it is exactly the distinction we just drew between consuming and holding:

- An equation has level 0. (Nothing to consume; contentless.)
- **and** and **or** take the level of their more complex side — packaging two realizers, or tagging one, adds no new layer of consuming.
- **An implication adds one to the level.** It is a procedure that consumes a realizer of its hypothesis, so it sits one layer above what it consumes.
- **A universal adds one to the level,** for the same reason: it consumes the number you instantiate at and returns a realizer, one layer up.
- **An existential adds nothing to the level.**

Why does $\forall$ cost a level but $\exists$ does not, when both mention a bound variable? Because $\forall y. \varphi$ is a *procedure* — you hand it a y and it works — while $\exists y. \varphi$ merely *holds* a y , the witness, already chosen. A universal consumes; an existential packages. Consuming costs a level; packaging is free. Look back at Act II: the witness 5 was already sitting in the realizer, waiting to be read; nothing had to be *run* to consume an input and produce it. That is exactly why our existential example was so simple: $\exists$ adds no level. This is not a rule imposed from outside — it falls straight out of the difference between a realizer that runs and a realizer that merely stores.

Where a realizer lives: **PureType**

We keep saying a realizer “is a number,” or “is a procedure taking a realizer and returning a realizer.” For the simplest cases that is an adequate description. But once realizers can be procedures whose inputs are procedures, we need to say precisely what mathematical objects these are — otherwise “a procedure taking a realizer” is a phrase, not a thing. Now that Act II has given us a concrete realizer to point at, and the level has told us how deep the nesting can go, we can place them.

Realizers live in a hierarchy of types called **PureType**, built up in layers by that same level count. The bottom layer, **PureType** at level 0, is just the natural numbers — this is where a bare witness like our 5 lives. The next layer up is the functions from the bottom layer to the numbers; the layer above that, functions from *those* to the numbers; and so on. Each time a formula’s level goes up by one — each time you add an implication or a universal that *consumes* a realizer — the realizer climbs one storey in this tower. A realizer of a level- n formula lives n storeys up.

This is why we spent so long on levels before naming **PureType**: the level is exactly the address, the floor number, telling you which storey of the tower a formula’s realizers occupy. Our Act II realizer was a level-0 object — a number — because its formula was an existential over an equation, and neither the equation nor the existential adds a level. Had the formula been, say, an implication whose hypothesis was itself an implication, its realizers would have lived two storeys up, as procedures that consume procedures. The tower is just bookkeeping for “how much consuming is going on,” made into actual mathematical objects a machine can build and evaluate.

One thing to carry forward, because Act V turns on it: this tower — the numbers, the functions between the layers, the operations for packing and unpacking pairs and applying one layer’s object to the layer below — is built by completely elementary means, with nothing classical or non-constructive anywhere in it.

The program a closed statement denotes: **CtQ**

There is a subtlety we have glossed. Two *different* proofs of the same statement can produce two *different* realizers — different objects in the **PureType** tower — even though both compute the same answers. If we want to speak of “*the* program” a theorem denotes, we need to know when two realizers should count as the same program.

The answer is: when they agree as functions — when, fed the same inputs, they always return the same outputs, all the way up and down the tower. Collapsing realizers that agree everywhere into single objects gives a cleaner space, the space of *extensional* functionals, and for a closed

statement (one with no free variables left dangling), its realizer lands in a specific such space the development calls **CtQ**. The single element of **CtQ** a closed statement’s realizer collapses to is, in the strongest sense, *the program that statement denotes* — proof-independent, blind to how you happened to prove the theorem.

We name **CtQ** here but do not yet lean on it, because it needs one further ingredient we have not established: for the collapse into extensional functionals to be the *right* space — the honest, well-behaved space of type-2 functionals rather than an arbitrary quotient — the realizers landing in it must be **continuous**, a property we are about to state. So read **CtQ** for now as a promissory note: *the program of a closed theorem is a single, proof-independent object, once we know its realizer is continuous*. The next two subsections supply, respectively, the guarantee that **extract** is correct at all, and then that continuity.

Soundness: it works for every proof, not just ours

Now the second of this act’s two opening questions. We ran **extract** on one proof and got a correct realizer. Is that always so?

It is, and it is a theorem — the central theorem of the whole construction. It is called **soundness**, and it says:

For every proof the fragment can build, of every statement, **extract** returns a genuine realizer of that statement.

Read against Act II: what we watched happen once, by hand, for $\exists s. \text{good}(3, s) = 0$, soundness guarantees happens for *every* derivation the fragment admits. The witness always comes out; the certificate always certifies; the pair always has the shape the connective demands.

The proof of soundness is itself an induction — not over numbers, but over the shape of proofs. There are finitely many inference rules in the fragment. Soundness checks, once and for all, that each rule’s little piece of **extract** does the right thing *assuming the pieces for the rule’s sub-proofs already did*. Because every proof is a finite tree built from those rules, checking each rule once covers every proof there will ever be. This is the same move as ordinary induction — verify a base case and a step and you have covered infinitely many numbers — lifted from numbers to proof-trees.

Soundness is what upgrades the Act II demonstration from an anecdote into a guarantee. From here on, whenever we prove a theorem in the fragment, we may speak of “the program it extracts to” and know, without further checking, that the program is correct: it computes something that genuinely satisfies the theorem’s statement, at every input.

Generic continuity: every program is continuous

One more property, stated now and explained more deeply once Act IV has given us programs worth explaining it for.

A realizer of a closed statement can be a functional — an object high in the **PureType** tower that takes functions as inputs. There is a classical worry about such objects: a functional that reads *infinitely much* of its input before deciding what to return is a pathological thing, and it is exactly the kind of object that does not collapse honestly into the extensional space **CtQ** we

described. The good functionals — the ones **CtQ** is really about — are the **continuous** ones: those that, to determine any single output, consult only *finitely much* of their input.

The development proves, once and generically:

Every realizer that **extract** produces, from every proof, is continuous.

This is what redeems the promissory note on **CtQ**. Because every **extract** is continuous, every closed theorem’s realizer collapses honestly into a single element of **CtQ** — a single, proof-independent program. We will see in Act VI *why* this generic continuity holds — it turns on a small and rather satisfying fact about how the extraction machinery is built — but the harder examples of Act IV should exist first, to make that “why” worth the space. For now: **extract** is correct (soundness) and its output is continuous (generic continuity), so every theorem of the fragment denotes one honest program.

The box is open. We have the general notion of realizer, the level accounting, the tower they live in, the collapse into programs, the guarantee that extraction is correct, and the guarantee that its output is continuous. Everything from here runs on exactly this machinery, unchanged. What varies is only the proofs we feed it — and each of the next several asks the machine to do one thing it has not done before.

Act IV — Running the same machine on harder proofs

Nothing new gets added to the extractor in this act. The pipeline — prove a theorem, run **extract**, get a certified program — is fixed. What changes is the difficulty of the proofs, and each example below is chosen because it forces exactly one genuinely new capability out of the same fixed machine. We name that capability directly, in a heading, so the staircase is visible.

Before the first step, one rule the first five examples all lean on.

The rule that becomes recursion: ordinary induction

The fragment has ordinary mathematical induction as one of its inference rules — call it **ind**. It says the familiar thing: to prove $\forall x. \varphi(x)$, prove $\varphi(0)$ and prove that $\varphi(x)$ implies $\varphi(x + 1)$.

Watch what **extract** does to a proof that uses **ind**. The base case $\varphi(0)$ extracts to some realizer — the answer at 0. The step extracts to a *transformation* — a procedure turning a realizer of $\varphi(x)$ into a realizer of $\varphi(x + 1)$. To get the realizer at a particular number k , **extract** starts from the base and applies the step k times. That is precisely a **recursive program**: base value, and a function iterated as many times as the input says. A proof by induction, run through **extract**, is a program by recursion. This is the shape underneath all five of the next examples; each supplies a different base and step, and so extracts to a different recursive program.

New capability: a structured witness (Towers of Hanoi)

The Towers of Hanoi puzzle asks for a sequence of moves that transfers a stack of n disks from one peg to another. The fragment can state, and prove, that such a sequence always exists — and, in the same breath, that it has exactly $2^n - 1$ moves.

What is new here is the *shape of the witness*. In Act II the witness was a bare number, 5. Here the thing whose existence we prove is a whole *sequence of moves* — a structured object, encoded as a number but meaning a list. When `extract` runs on this proof, the realizer’s witness is not a count but the actual move sequence, and it can be decoded back into a readable list of “move a disk from this peg to that peg” instructions — the classical optimal solution, produced by the extracted program and checked to be exactly the moves a human would write down. The existential machine of Act II, unchanged, now hands back structured data rather than a single number. (The proof itself uses `ind` on the number of disks, so by the previous subsection it extracts to a recursion; the recursion branches, solving two smaller sub-towers and joining them with one move between — which is why the move count comes out $2^n - 1$.)

New capability: a decision procedure (Pascal’s triangle, mod 2)

Take Pascal’s triangle and replace every entry by its remainder on division by 2 — every number becomes a 0 or a 1. The fragment proves, for every position in the triangle, that its parity is either 0 or 1 (an utterly unsurprising statement — every number is even or odd) and, more to the point, *which*.

The new capability is that the extracted program is a **classifier**. Recall the realizer of a disjunction φ or ψ : a tag saying which side holds. The statement here is a disjunction — “this entry is 0, or it is 1” — and so its realizer carries a tag: for each position, a bit naming the answer. Run `extract` and that tag becomes a runnable function from a position to its parity — a genuine decision procedure, produced not by us writing a classifier but by extracting one from a proof that merely said “the answer is one or the other.”

And when you tabulate that extracted classifier’s outputs — draw a # for 1 and a blank for 0, row by row — the picture that appears is the Sierpiński triangle, the familiar fractal of nested holes. There is no gap between the picture and the theorem: every cell of the drawing is one instance of the proved disjunction, its value filled in by the extracted realizer’s own tag. The fractal is not illustrating the theorem from outside; it *is* the theorem’s extracted program, run over a grid.

New capability: a strengthened induction hypothesis (Fibonacci)

The Fibonacci numbers 0, 1, 1, 2, 3, 5, 8, ... are defined by “each is the sum of the previous two.” Suppose we want to prove, in the fragment, that the Fibonacci function is total — that for every n there is a value `fib(n)` — in a form from which we can extract a program computing it.

Try to do this by ordinary induction and you hit a wall that is worth feeling. Ordinary induction gives you, at the step, the truth of the statement *at n* to work with. But computing `fib(n + 2)` needs *two* earlier values, `fib(n + 1)` and `fib(n)` — and the induction hypothesis at n alone does not hand you both. The naive statement does not go through.

The new capability is the standard cure, and watching it extract is the point. Instead of proving “`fib(n)` exists,” prove the *stronger* pair statement: “the *pair* (`fib(n)`, `fib(n + 1)`) exists.” Now the induction hypothesis at n hands you both consecutive values at once, and the step is a single shift: from (`fib(n)`, `fib(n + 1)`) you produce (`fib(n + 1)`, `fib(n + 2)`) by keeping the second component and adding the two together. Strengthening what you prove makes the

induction go through, because a stronger hypothesis is a stronger thing to be handed at the step.

Extract that proof and — exactly as the “induction becomes recursion” subsection predicts — you get the fast, two-variable iterative Fibonacci program every programmer knows: carry a pair, shift it forward n times, read off a component. The proof’s strengthened hypothesis *is* the program’s pair of accumulator variables. You can watch the extracted realizer’s pair advance through $(0,1)$, $(1,1)$, $(1,2)$, $(2,3)$ — the loop’s running state, read straight out of a proof that never mentions a loop.

New capability: taming variable capture (Euclid’s gcd)

The fragment can prove that any two numbers have a greatest common divisor — and the extracted witness *is* the gcd, computed by subtractive Euclid (repeatedly replace the larger number by the difference), with no gcd operation built into the fragment at all. Two supporting ideas make this possible, and both show the fragment’s economy. First, the fragment never needed to add an “order” operation to talk about one number being smaller than another: $s < t$ is simply *there exists a d with $\text{succ } s + d = t$* — order is an existential over addition, not a new primitive. Second, the fragment never needed a separate “strong induction” (course-of-values induction, where the step may use *all* smaller cases, not just the immediately previous one): it is *derived* from ordinary *ind*.

But the new capability the gcd proof forces is neither of these — it is a problem about *substitution*. When you instantiate an induction hypothesis, or apply a “for all” statement, you substitute a specific term for a bound variable. If that term happens to mention a variable that is *also bound* somewhere in the formula you are substituting into, the substitution silently confuses two different variables that share a name — the notorious *variable capture* of formal logic. The fragment’s substitution is deliberately simple-minded: rather than quietly renaming to avoid capture, it *forbids* the capturing substitution outright. So the burden falls on the proof author to route around capture, and Euclid’s recursion is the first place two different capture problems bite at once.

The fragment has two general-purpose dodges for this, developed on earlier theorems, and the gcd proof is the first to use them *together*. One dodge: when you need to instantiate an induction hypothesis at a term that mentions a bound variable, first give that term a fresh name with an extra “for all,” instantiate at the harmless *name*, and only then connect the name to the term. The other: when a recursion permutes its arguments so that you must instantiate a “for all” at a rearrangement of its own just-bound variables, rename those variables to fresh ones first. Neither dodge changes what is proved or what the extracted program computes; both are purely about getting a naive substitution system to accept a substitution that is morally fine but literally capturing. The gcd derivation composes the two, and out the far end comes a certified program that is a genuine greatest-common-divisor calculator — and, pleasingly, it needs no special case for zero: the specification is satisfied by $\text{gcd}(0, 0) = 0$, everything divides 0, and so no positivity precondition is required at all.

New capability: more general than the classical statement (Sperner, 1D)

Colour the points $0, 1, 2, \dots, n$ along a line, each some natural-number colour, with the two ends coloured differently. Then somewhere along the line two *adjacent* points must have different colours — you cannot get from one end-colour to the other without a change somewhere. This is the one-dimensional case of Sperner’s lemma, and the discrete ancestor of the intermediate value theorem: if a walk starts low and ends high, it crosses every level in between.

The fragment proves it by ordinary induction, scanning along the line and carrying “either we are still at the start colour, or we have already found a crossing.” The extracted program is a genuine left-to-right search that returns the *first* crossing.

But the new thing is not the extraction; it is the theorem. The classical statement is usually given for *two* colours — the ends are 0 and 1, and the crossing is a 0-next-to-1. The fragment’s proof never uses the restriction to two colours. It works for *arbitrary* natural-number colourings: any colours at all, ends merely required to differ. The extracted search is correspondingly more general than the classical picture. The proof turned out to prove *more* than the theorem it was written for — a small illustration that formalizing a statement faithfully sometimes reveals it was never really about the special case you first met.

New capability: recursion along the ordinals (Goodstein)

Now Goodstein itself, the general theorem behind Act II’s single hand-worked case.

Back in Act II we proved, by hand, that the Goodstein sequence from 3 reaches 0 — a single existential, $\exists s. \text{good}(3, s) = 0$, witness supplied by us as 5. Goodstein’s *theorem* is the universal version:

$$\forall m. \exists s. \text{good}(m, s) = 0$$

— for *every* starting value m , the sequence reaches 0 at some step s . This is the same shape of statement as Act II’s, one universal quantifier out front, and we want the same thing from it: a program that, given m , returns the stopping step. In Act II we knew the witness because we knew the answer for 3. Here the witness must be produced *uniformly*, by the proof, for a starting value we have not seen.

And this is where Act I’s promissory note comes due — the fact that ordinary induction is not enough. If you try to prove Goodstein by inducting on m , or on s , or on the value $\text{good}(m, s)$, you fail, because the value goes *up* at almost every step; there is no ordinary numerical quantity that decreases as the sequence runs, so there is nothing to induct downward on. The quantity that *does* decrease is not a natural number at all. It is an **ordinal** — a number from a system that extends the naturals past infinity in a controlled way, far enough to reach the height called ε_0 (epsilon-nought). Each Goodstein step, though it inflates the natural-number value, strictly *lowers* an ordinal assigned to the current state; and because that ordinal system is *well-ordered* — has no infinite descending chains — the sequence cannot descend forever, so it must terminate. This is the extra strength Act I warned you about: **transfinite induction up to ε_0** , a principle strictly beyond ordinary induction on $\mathbb{N}$. The fragment has it as one inference rule — the single rule in its whole repertoire that is not a consequence of ordinary logic and ordinary induction. It is the reason the fragment can prove Goodstein at all.

To make that rule real and mechanical, the fragment builds the ε_0 ordinals *as ordinary numbers under a hand-crafted encoding*, defines the “is a smaller ordinal than” relation on those codes, and proves — by elementary means, no infinite reasoning — that this relation is well-founded, i.e. admits no infinite descending chain. That well-foundedness proof is what licenses recursion along the ordinals. (There is a specific reason this substrate was built by hand rather than borrowed from a standard library, and it is bound up with Act V’s result; we defer it to Act VI, where it lands as a discovery rather than a digression.)

Now watch the two acts line up. In Act II, `extract` turned an existential proof into a witness by reading a number the proof supplied. Here, `extract` turns Goodstein’s proof into a witness by *recursion along the ordinals*: the transfinite-induction rule extracts to a recursive program that descends the ordinal assigned to each Goodstein state until it reaches the bottom, and the number of descents is the stopping step. Where `ind` extracted to “iterate a step as many times as the input says” (an ordinary recursion), the transfinite rule extracts to “recur along the ordinal ordering” — a genuinely stronger recursion, matching the genuinely stronger induction. Same machine, one gear higher.

And the program runs. Ask it for the stopping step of the Goodstein sequence starting at `m` and it computes:

<code>m:</code>	<code>0</code>	<code>1</code>	<code>2</code>	<code>3</code>	<code>...</code>	<code>4</code>
<code>stopping step:</code>	<code>0</code>	<code>1</code>	<code>3</code>	<code>5</code>	<code>...</code>	<code>(not attempted)</code>

The entries for `m = 0` and `m = 1` the extracted program returns directly when you evaluate it — 0 and 1. The stopping steps 3 and 5 for `m = 2, 3` are the lengths of the Goodstein sequences 2, 2, 1, 0 and 3, 3, 3, 2, 1, 0; soundness guarantees the extracted program returns exactly those, and the fragment’s own `good` operation computes them, even though running the extracted program itself that far is slow (a cost we examine in Act VI). And `m = 4` is simply not attempted: its Goodstein sequence begins 4, 26, 41, 60, ... and climbs into stopping steps too astronomically large to reach. This last row is the moment Act I’s “numbers explode” stops being an assertion. You are watching a theorem’s extracted program decline to finish a computation because the theorem is *true but enormous* — the terminating sequence really is that long.

New capability: a fragment theorem versus a metatheorem (the Hydra)

The last and hardest example marks a boundary — the place where what the fragment can *prove* and what it can even *state* come apart.

The Hydra is a many-headed tree. Hercules fights it by chopping a head; each chop makes the Hydra regrow — often growing *more* heads than it lost, sometimes far more. It looks unwinnable. The Kirby–Paris theorem says Hercules wins anyway: every battle, however the regrowth goes, terminates with the Hydra dead. The reason is Goodstein’s reason in disguise — each chop, though it grows the tree, strictly lowers an ordinal below ε_0 assigned to the tree, and ordinals cannot descend forever.

The fragment can prove termination for *one fixed, encoded battle*: fix a starting Hydra and a fixed way of playing, code the whole thing as numbers, and prove $\forall h. \exists t. \text{hydra}(h, t) = 0$ — for every starting Hydra `h`, there is a step `t` at which the battle has ended. This is the exact

shape of Goodstein’s theorem, proved the exact same way, transfinite recursion and all, and it extracts to a program computing the battle length. So far, business as usual.

But the *real* Kirby–Paris theorem says more: Hercules wins *no matter what strategy* he uses and *no matter how* the Hydra regrows. And this fuller statement the fragment **cannot even express**. To say “for every strategy,” you must quantify over strategies — and a strategy is a *function* (it looks at the current Hydra and decides the next chop). The fragment’s “for all” ranges only over *numbers*, never over functions. It has no way to write “for all strategies” at all. The limitation is not that the fragment fails to *prove* the general theorem; it is that the sentence expressing the general theorem is not in the fragment’s language.

So the general theorem is proved *outside* the fragment — in the ordinary mathematics of the proof assistant itself, which can freely quantify over functions. There, one proves that *every* legal Hydra move, by any strategy, lowers the same ordinal, and hence every battle whatsoever is finite; and one checks that the fragment’s single fixed battle is one of these general plays, so the fragment’s theorem is a genuine special case of the general one, not a different game. This split — a *fragment theorem* for one encoded battle, a *metatheorem* for all strategies — is forced, cleanly and provably, by the single fact that the fragment has no function variables. It is a boundary that mathematics draws for you, rather than one you choose, and it is the right place to stop feeding the machine harder proofs.

Act V — What it costs: the constructivity audit

We have now watched proofs of real strength become running programs. Goodstein and the Hydra needed transfinite induction up to ε_0 — genuinely more than ordinary arithmetic. And the programs we extracted are not toys: an ordinal-descending recursion, a first-crossing search, a greatest-common-divisor calculator, a parity classifier.

So here is the suspicion any careful reader should now be holding, and it is the right suspicion. Proofs of this strength, in ordinary mathematical practice, routinely use *non-constructive* reasoning — the law of excluded middle, the axiom of choice, arguments that assert an object exists without building it. Surely, somewhere in a construction reaching all the way to ε_0 , some non-constructive step sneaks in. And if it does, then the extracted “program” might be a fiction — an object that type-checks but cannot actually run, its computational content secretly resting on an axiom that computes nothing. The whole promise of the tour would collapse at exactly its most impressive point.

This act resolves the suspicion, precisely, in the program’s favour. And the resolution is possible only because the proof assistant tracks, mechanically and unforgeably, exactly which axioms any given object depends on. Ask it about any definition or theorem and it reports the complete list.

The suspicion has a specific name. The genuinely non-constructive axiom — the one that asserts choices can be made without giving any rule for making them, and from which the full law of excluded middle follows — is called **Classical.choice**. It is the axiom whose presence in a program would hollow the program out, turning a thing that computes into a thing that merely

type-checks. So when we ask whether something non-constructive rode into the extracted programs, the question has one precise form: does `Classical.choice` appear in the list?

Now the audit’s finding, stated exactly as the machine reports it.

Every extracted program in the whole development — the Goodstein stopping-step function, the Hydra battle-length function, the gcd calculator, the Sperner search, the Pascal classifier, the Fibonacci iterator, and the extraction machinery itself — reports its axiom dependency as exactly

[`propext`, `Quot.sound`]

and `Classical.choice` is not on the list. Not “we believe”; not “morally.” The kernel of the proof assistant, asked directly, lists those two names and not the third, for every object that actually runs.

So what *are* the two names that are there? Neither is the one the suspicion feared. **`propext`** and **`Quot.sound`** are the proof assistant’s own two foundational axioms, and both are constructively harmless: neither lets you conjure an object you cannot build, neither implies the law of excluded middle. They are bookkeeping about when two propositions, or two quotient elements, count as equal — part of the machine’s floor, computationally inert. The trusted base of every extracted computation is the proof assistant’s kernel together with these two, and nothing classical.

The suspicion is not merely soothed; it is *refuted, mechanically*. The recursion up to ε_0 that powers Goodstein and the Hydra does not smuggle in choice, because the ε_0 substrate — the ordinal codes, the smaller-than relation, the well-foundedness proof licensing the recursion — was built, deliberately, without any classical step. (That deliberateness is the Act VI discovery we have twice deferred.)

Does `Classical.choice` appear *anywhere* in the project? Yes — and where it appears is a precise boundary. It enters in exactly two places, and neither of them runs.

- It appears in some of the *correctness proofs* — the proofs that the extracted programs meet their specifications — and even there only through *two named, humble lemmas* from the standard library, about a routine operation for updating a function at a point. These lemmas are about equality of functions; they live entirely in the realm of propositions; they are used to *prove that a program is correct*, never inside the program. A program’s correctness proof may lean on a classical equality-of-functions fact without one atom of that fact entering the program’s runtime behaviour. (And these two lemmas are, in fact, constructively true; reproving them by hand would remove even this trace. The dependency is a convenience, not a necessity.)
- It appears in the final *packaging* step — the collapse of a realizer into the single extensional program-object of CtQ from Act III. That collapse is a semantic act of identification, performed once to say “here is the one program this theorem denotes,” and it inherits choice from the general theory of extensional functionals. But again: the identification is *about* the program; the program itself is the choice-free realizer that got identified.

Draw the picture and it is stark. There is a wall. On one side, everything that *runs* — every extracted program, the entire executable pipeline — and that side is free of `Classical.choice`

entirely, resting only on the inert kernel axioms. On the other side, some *proofs about* those programs, and one *packaging* step, which may use choice freely because nothing on that side ever executes. The one non-constructive axiom in the whole development is quarantined on the far side of the wall from anything that computes.

A proof needing strictly more than Peano arithmetic — transfinite induction to ε_0 — became a program, and the program is *constructive to the last axiom the machine can name*. The strength went into proving the theorem; none of it leaked into the thing that runs.

Act VI — What Lean forced us to discover

Some of what we have seen was designed. But some of it was not chosen at all — it was *found*: facts that turned out to be true and had to be respected, boundaries that mathematics drew rather than the authors. This act collects those, because they are the places where formalizing something taught its authors something they did not set out to learn.

Why the level accounting has the shape it does. We introduced, in Act III, the rule that $\forall$ and $\rightarrow$ cost a level while $\exists$, $\wedge$, $\vee$ cost none, and justified it by “consuming versus packaging.” Seen as a discovery rather than a rule: the level system did not have to come out this clean, and the fact that the *only* things that cost a level are the two connectives whose realizers actually run — the procedures that consume an input — is a genuine regularity of realizability, not a design decision. Binders that consume realizers cost a level; binders that merely store them are free. The existential’s freedom, in particular, is why witness-extraction stayed as cheap as the disjunction it is built from — and that cheapness is what let the same simple machinery of Act II carry all the way up to Goodstein.

Why the Hydra split was forced. In Act IV we found that the fragment can prove one encoded Hydra battle terminates but cannot state “Hercules wins for every strategy,” because a strategy is a function and the fragment quantifies only over numbers. This is the clearest case in the whole tour of a discovery masquerading as a design choice. Nobody decided the fragment “should not” prove the general theorem; the general theorem’s *sentence* simply is not writable in a language without function variables. The fragment/metatheorem division is not a limitation the authors imposed to keep things simple — it is a fact about the expressive reach of the language, which the attempt to state Kirby–Paris in full ran straight into.

Why the ε_0 substrate had to be hand-rolled. Here is the discovery Act V twice pointed at. The proof assistant’s standard library already contains a way of coding pairs of numbers as single numbers — exactly the kind of coding the ε_0 ordinal notations need. The obvious move is to reuse it. It cannot be reused. That standard pairing, and every lemma the library proves about it, depends on `Classical.choice`. Had the ε_0 substrate been built on it, `Classical.choice` would have flowed straight into the definition of the transfinite recursion — and from there into every extracted program that uses transfinite recursion, which is Goodstein and the Hydra. The clean `[propext, Quot.sound]` verdict of Act V would have been *false*, and not fixably so, because the choice would have been welded into the recursion itself rather than sitting off in a correctness proof. So the ordinal codes, the ordering, and the well-foundedness proof were all built from scratch by elementary means, specifically so that no classical axiom touches the

machinery that runs. The constructivity result of Act V is not a happy accident discovered at the end; it is a constraint that reached backward and dictated how the foundations had to be laid. The audit was, in effect, being passed before it was run.

A fully verified optimization that changes nothing. The transfinite recursion that powers Goodstein and the Hydra is slow to *evaluate* — slow enough that the extracted programs finish only for small inputs, as we saw with Goodstein’s $m = 4$. The natural fix is memoization: remember the recursion’s result at each ordinal so it is never recomputed. This was built, and *fully verified* — proved to compute the same answers as the un-memoized version. And then it was measured, and it saved nothing. Instrumentation showed the memo table being consulted thousands of times, finding the same entry over and over, and the wall-clock time was unchanged within noise. The reason is a genuine negative result, and worth stating because it corrects the obvious intuition: the recursion’s value at an ordinal is not expensive to *compute* — it is produced in a single step, as a small closure — so there is nothing to save by remembering it. The real cost is elsewhere entirely: in how many times that closure gets *applied* as the surrounding program runs, which no table keyed by the ordinal can reach, because it is not recomputation at all. The optimization was correct and useless, and finding out *why* it was useless — measured, not guessed — is a fact about where the cost of this kind of extracted program actually lives.

A fact that moved from assumed to derived. Early in the development, the crucial descending step of the Goodstein proof — that one Goodstein step really does lower the ordinal — was taken as an *assumption*, an extra rule handed to the fragment. That made the proof honest but unsatisfying: the fragment proved Goodstein’s theorem only *relative to* the very fact that makes it true. Later, that assumption was *derived* instead, broken into three much smaller assumptions — each a single, general property of one operation, none of which so much as mentions the Goodstein sequence — from which the descending step follows by the fragment’s own reasoning. The gap did not close entirely (those three small properties are still assumed rather than proved inside the fragment), but it narrowed from “assume the thing you are trying to prove” to “assume three general facts about arithmetic and derive the rest.” Watching a formalization tighten its own hypotheses over time, turning an assumption into a consequence, is the kind of bookkeeping honesty that only a machine-checked development makes fully visible.

Act VII — Closing

One pipeline handled all of these — a Goodstein stopping-step function, a Hydra battle length, a gcd, a parity classifier, a first-crossing search, a Fibonacci iterator — from proofs that ranged from ordinary induction all the way up to transfinite recursion. That is true. But that proofs contain programs is an old idea; nobody should be surprised, in the abstract, that a constructive proof of “something exists” can be made to cough up the something. What this tour exhibits instead is how few moving parts it took to make that real, across such a range.

A realizer is a witness, or a pair, or a tag, or a procedure — four shapes. A level counts how much consuming a formula’s realizers do. `extract` walks a proof and, at each rule, does the one

small thing that rule calls for; soundness checks each rule once. Ordinary induction extracts to ordinary recursion; transfinite induction extracts to recursion along the ordinals; an existential extracts to its witness. Hanoi’s branching, Pascal’s parity, gcd’s substitution dance, Sperner’s scan, Goodstein’s ordinals, the Hydra’s boundary — all of it ran through that same handful of parts, unchanged, and came out as programs whose every runtime axiom the machine can name, none of them classical.

The same four shapes, the same level count, the same proof-walking translation carried from the one hand-built existential of Act II — $\exists s. \text{good}(3, s) = 0$, a single witness read back as 5 — to the edge of what a language without function variables can say.

Everything above has a fully formal counterpart — the actual Lean development, every theorem machine-checked, every axiom dependency the kernel’s own verdict, and a technical write-up that states each claim in its exact form with its exact hypotheses. This tour is the map; that is the territory. If any sentence here made you want to see precisely how it is done rather than merely that it is, that is the right instinct, and the formal development is where it is answered.

— *end* —